

[← Back to Discussion](#)

LH24 • Posted 2 日前

## Sharing a 0.708 public LB submission and clean, modular code with

Hello everyone, I went quickly over the data, and I thought it would be useful if I shared how I usually structure my experiments, and some quick experiments and eda. Please upvote if you find this topic helpful.

**Important Note:** This topic reflects my quick analysis and experimenting with the data. I put some assumptions that may be right and they may be debatable or reflect only this experiment. Take what is useful for you. Happy reading!

### The Task

The goal is to predict a 41×41 pixel GPM-IMERG precipitation map (mm/hr) 30 minutes ahead, using up to 30 minutes as input. Each satellite image has 16 spectral bands at resolutions ranging from 81×81 (Himawari) to 1024×1024 (GOES). The model must match the target.

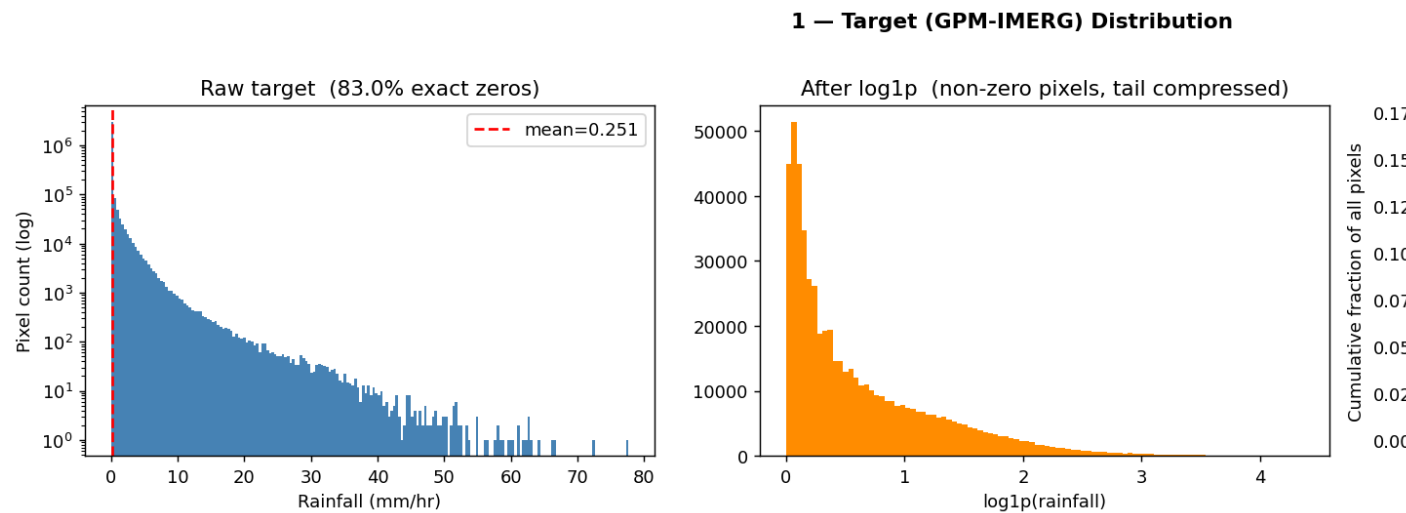
So we need to put a few things in mind...

- **83% of target pixels are exact zeros (training-set statistic).** The distribution is extremely zero-inflated (mean = 0 mm/hr, max = 77.6 mm/hr across training samples). RMSE is driven almost entirely by rare heavy rain events.
- **Geographic holdout.** Train and test locations are disjoint by design. A model that memorizes regional patterns will fail on unseen locations. As one data point: mean GPM rainfall per location varies by orders of magnitude between the wettest (Bangladesh) and the driest (dhaka, Bangladesh) in our training-sample analysis. I recommend location-based cross-validation to estimate generalization here.
- **Three different satellites.** Himawari, GOES, and Meteosat each cover different parts of the world. For each, you cannot use raw band numbers across satellites without a per-satellite mapping.

## 2. Let's talk about the baseline

|                            |        |                             |
|----------------------------|--------|-----------------------------|
| WV upper (6.2 $\mu$ m)     | -0.221 | Upper-tropospheric moisture |
| Visible red (0.64 $\mu$ m) | +0.129 | Daytime cloud reflectance   |

The negative sign for IR bands is physically meaningful: colder cloud tops indicate higher, deeper convection. Visible and near-IR bands have a small positive correlation (brighter cloud = some rain) that is much weaker at night, where visible is irrelevant regardless of rain.



The raw distribution (left) is nearly all zeros with a heavy right tail. After log1p transform (centre, non-zero), the distribution is more symmetric, and that extreme rainfall values no longer dominate the gradient. The CDF (right) shows that 99% of non-zero pixels are below 5 mm/hr, but pixels above 5 mm/hr dominate the RMSE because errors scale quadratically.

### Trivial baselines (raw mm/hr RMSE):

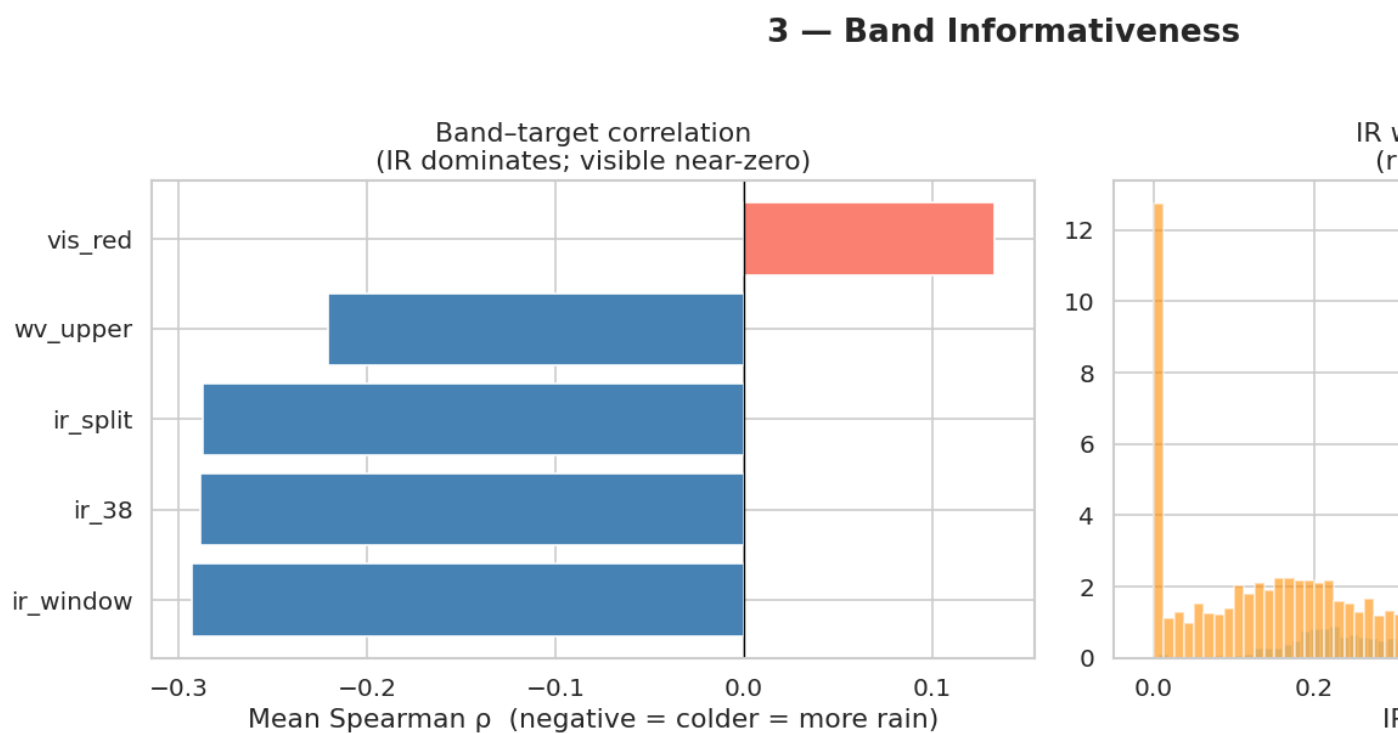
- Predict zero everywhere: **1.322**
- Predict the training mean everywhere: **1.298**
- Official CNN (visible bands, MSE): **0.913**

The 0.913 figure is the official CNN baseline on the public leaderboard (visible bands, MSE loss). Our holdout score is comparable to leaderboard scores since they come from a different evaluation split, but they are useful for comparison with each other under identical conditions.

## 3.2 Geographic Diversity

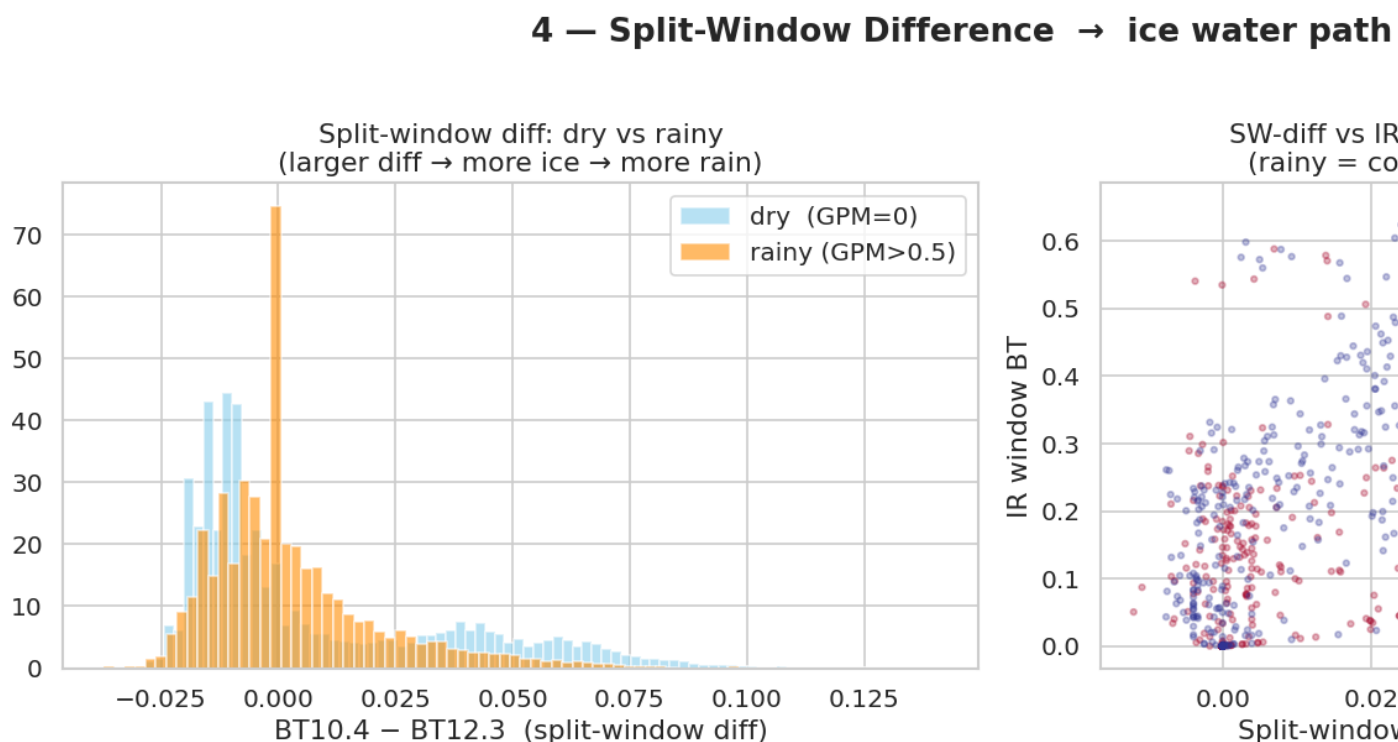
### 2 — Geographic Diversity → why LOO-CV is essential

### 3.3 Band Informativeness

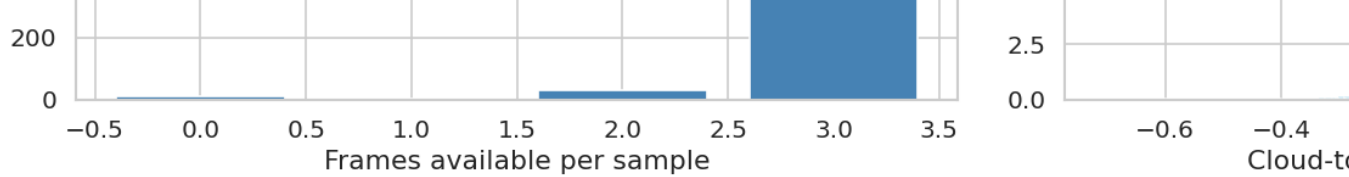


The left panel shows Spearman  $\rho$  with the GPM target across 1,500 samples. IR bands cluster around  $\rho = -0.25$ , which is driven by daytime samples, at night the signal drops substantially. The right panel shows the IR window BT distribution split by dry vs rainy pixels, we can see a clear and consistent shift..

### 3.4 The Split-Window Difference



The BT10.4 - BT12.3 split-window difference is a useful physical proxy related to cloud microphysics. At 10.4 and 12.3  $\mu\text{m}$ , the difference in brightness temperature is related to the ice water path (IWP) and thus to the amount of ice in the cloud.



97.5% of training samples have all three time frames. The cloud-top cooling rate ( $BT_{t0} - BT_{t2}$ , measured in  $^{\circ}C$  per pixel), intensifying convection (getting colder) precedes heavy precipitation. Note that the three temporal frames capture the convective tendency.

## 4. Model

I use a small encoder-decoder U-Net with skip connections. Input: 9 channels ( $3 \text{ spectral bands} \times 3 \text{ time frames}$ ). Output: 1 channel (precipitation map).

Input (9, 41, 41)

→ Encoder: Conv  $9 \rightarrow 32 \rightarrow 64 \rightarrow 128$

→ Bottleneck: Conv  $128 \rightarrow 256$

→ Decoder: Transposed conv with skip connections  $256 \rightarrow 128 \rightarrow 64 \rightarrow 32$

→ Head: Conv  $32 \rightarrow 1$

Output (1, 41, 41)

Total parameters: **1,928,737**. This is intentionally small, fast to train, easy to understand, and the signal is clear once representation learning once the bands are chosen well.

The model pads  $41 \times 41$  inputs to  $48 \times 48$  (divisible by 8 for 3 pooling levels) and crops back after the decoder.

**Loss function:** I train on MSE in  $\log_{1p}$  space:

```
1 loss = MSE(model(x), log1p(y))
2 pred_mm_hr = expm1(model(x)).clamp(0)
3
```

At inference I invert with  $\expm1$  and clip to zero. All RMSE values reported are in raw mm/hr. This re-weighting is done without changing the evaluation metric.

5% of the total because a 50 mm/hr error contributes 2,500 more to MSE than a 500 mm/hr error. This is giving the model a stable learning signal. The right panel confirms that raw mm/hr baselines (predict-zero) are more lenient than their log1p counterparts, training in log space forces the model to predict structure rather than

## 5. Code Structure

```
1 discussion/
2 |─ config/
3 |   └─ train.yaml # all hyperparameters in one place
4 |─ models/
5 |   └─ init.py # build_model() registry
6 |   └─ unet.py # scratch U-Net (registered as "unet")
7 |   └─ timm_unet.py # any timm encoder + our decoder (registered as "timm_unet")
8 |─ dataset.py # data loading, band selection, normalisation
9 |─ train.py # training loop with Hydra + W&B
10 |─ generate_submission.py # inference → submission.zip (averages fold models)
11
```

### 5.1 Why Hydra

I recently started using Hydra, and it has made experiment management much easier, every experiment

```
1 config/train.yaml
```

. You override any value from the command line without touching code:

```
1 # Train with split-window bands, holdout validation
2 python train.py bands.combo=split_window data.cv_mode=holdout
3 # Compare MSE vs log1p loss on same bands
4 python train.py bands.combo=ir_classic training.loss=mse
5 python train.py bands.combo=ir_classic training.loss=log1p
6 # Switch to a ResNet18 encoder
7 python train.py model.name=timm_unet model.encoder=resnet18 model.pretrained=false
8 # Use mixed precision
9 python train.py training.precision=bf16
10
```

Hydra automatically saves a copy of the resolved config alongside each run's output. When you come back

exactly what parameters produced it.

```

14 model:
15     name: unet # "unet" or "timm_unet"
16     in_ch: 9
17     base_ch: 32
18 training:
19     epochs: 20
20     batch_size: 32
21     lr: 1.0e-3
22     weight_decay: 1.0e-5
23     patience: 5
24     loss: log1p # "log1p" or "mse"
25     precision: fp32 # "fp32", "fp16", "bf16"
26     seed: 42
27 bands:
28     combo: split_window # see dataset.py BAND_COMBOS
29 wandb:
30     entity: your-entity
31     project: solafune-nowcasting
32

```

## 5.2 Why Weights & Biases

I actually use it to organize my experiments and track them cleanly, W&B lets you put all runs on the same dashboard in a results table.

For these experiments, every run logs

```
1 val_rmse , train_rmse, train_loss and lr
```

at each epoch with the same metric names. In the W&B dashboard you can compare different experiments, view details and artifacts.

Setting up a run is three lines:

```

1 import wandb
2 wandb.init(project="solafune-nowcasting", config=OmegaConf.to_container(cfg))
3 wandb.log({"val_rmse": val_rmse, "train_loss": train_loss, "epoch": epoch})
4

```

## 5.3 Band Combinations

Five band combinations are implemented in dataset.py:

## 5.4 Cross-Validation

The training script supports two modes:

**Holdout** (cv\_mode: holdout): fix 4 locations as a permanent validation set. Fast, deterministic, comparable for band comparison because all models see identical val data.

**K-Fold** (cv\_mode: kfold): rotate through all locations in 5 folds. More reliable estimate of generalization on 16 locations and validates on 4 different ones. The submission averages predictions from all 5 fold models.

To generate a submission from all fold checkpoints:

```
1 python generate_submission.py \
2     --results_dir results/split_window_unet \
3     --bands split_window --model unet --loss log1p \
4     --output submission.zip
5
```

The script loads each

```
1 fold*_best.pth
```

It runs inference, averages the raw mm/hr predictions, and writes the submission zip.

## 6. Results

### 6.1 Band Comparison

All models: U-Net (1.9M params), log1p loss, early stopping (patience=5), Adam lr=1e-3. Validated on the s

U-Net val RMSE by band combination  
(holdout: florida / france / iakarta / kinchasa)

All numbers below are holdout val RMSE (4 fixed locations: florida, france, jakarta, kinshasa), not public l

| Band combo                  | Val RMSE (holdout) |            |
|-----------------------------|--------------------|------------|
| Visible (official approach) | 1.2857             | – (referen |
| IR classic                  | 0.8859             | –0.399 lov |
| WV moisture                 | 0.8825             | –0.403 lov |
| Ice proxy (3.9μm)           | 0.8809             | –0.405 lov |
| Split-window                | 0.8724             | –0.413 lov |

All IR combinations outperform the visible holdout baseline by a wide margin. The split-window combina over ir\_classic (0.8724 vs 0.8859, about 0.014 absolute) is modest but consistent – it may come from the making ice-water-path information more directly accessible to the model. The large gap between visible with visible bands having much weaker correlation with precipitation, especially at night.

The W&B runs table (below) shows that every experiment used identical hyperparameters, batch size 32, fp32 precision, so differences in val RMSE are attributable entirely to band choice, not configuration drift

Workspace

Runs

Automat.

Sweeps

Reports

Artifacts

Runs 3

Search runs

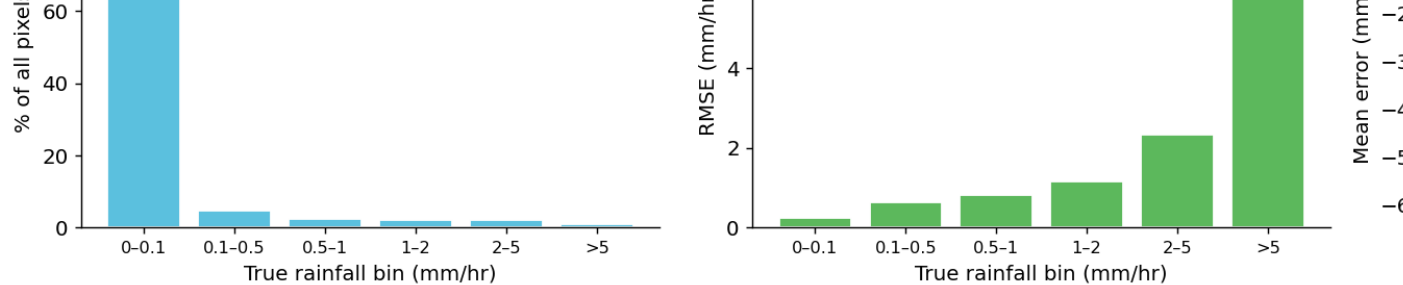
FilterGroupSortNew sweep

| <input type="checkbox"/> | <input type="checkbox"/>         | NAME 6 listed          | PATHS.OUTF        | TRAINING.B. | TRAINING.EI | TRAINING.LC | TRAINING.LI | TRAINING.N |
|--------------------------|----------------------------------|------------------------|-------------------|-------------|-------------|-------------|-------------|------------|
| <input type="checkbox"/> | <input checked="" type="radio"/> | ice proxy ...n+WV+IR]  | results/ice_prox  | 32          | 20          | log1p       | 0.001       | 4          |
| <input type="checkbox"/> | <input checked="" type="radio"/> | WV moist.../.3μm+IR]   | results/wv_mois   | 32          | 20          | log1p       | 0.001       | 4          |
| <input type="checkbox"/> | <input checked="" type="radio"/> | split-winc...ff+IR+WV] | results/split_wir | 32          | 20          | log1p       | 0.001       | 4          |
| <input type="checkbox"/> | <input checked="" type="radio"/> | IR classic .../12.3μm] | results/ir_classi | 32          | 20          | log1p       | 0.001       | 4          |

The training curves (below) put all four IR runs on the same chart. The val\_rmse panel shows early instab before recovering) followed by convergence to the 0.87–0.89 range by epoch 10–12. train\_rmse decrease and split-window tracking slightly below IR classic throughout. The train\_loss curves confirm all four run <0.01 by the end of training.

Filter





Breaking down the split-window model's errors by true rainfall intensity:

| True rainfall (mm/hr) | % of pixels |       |
|-----------------------|-------------|-------|
| 0-0.1                 | 88.1%       | 0.230 |
| 0.1-0.5               | 4.7%        | 0.630 |
| 0.5-1.0               | 2.3%        | 0.814 |
| 1.0-2.0               | 2.1%        | 1.162 |
| 2.0-5.0               | 2.0%        | 2.322 |
| >5.0                  | 0.9%        | 7.968 |

The pattern is consistent across all models and is a known limitation of per-pixel regression on zero-inflated data.

1. **Light rain (0-0.5 mm/hr):** the model overpredicts (positive bias). It produces a thin drizzle even at the boundaries.
2. **Heavy rain (>1 mm/hr):** the model underpredicts (negative bias). Extreme rainfall events are rare, so the model is biased towards the mean. The RMSE for pixels with >5 mm/hr true rainfall is 7.97 mm/hr, 9× the overall val RMSE.

The heavy-rain tail is where the competition is decided. 0.9% of pixels with RMSE=7.97 contributes disproportionately to the overall error, not the 88% zero pixels.

## 6.3 Per-Satellite Performance

### Per-satellite performance breakdown

|          |       |       |
|----------|-------|-------|
| Himawari | 1.126 | 68.0% |
| GOES     | 1.165 | 75.9% |
| Meteosat | 0.757 | 89.3% |

Meteosat achieves lower RMSE partly because its geographic coverage in the validation set (france, kins pixels. Lower baseline activity = easier for the model to predict. Himawari (jakarta) and GOES (florida) cov where heavy convection is more frequent and harder to predict.

## 7. Reproducing This

```

1 # 1. Clone and install
2 pip install torch torchvision rasterio hydra-core omegaconf wandb timm scikit-learn
3
4 # 2. Train split-window model with holdout validation
5 python train.py bands.combo=split_window data.cv_mode=holdout
6
7 # 3. Train all 5 band combos and compare in W&B
8 for COMBO in visible ir_classic split_window wv_moisture ice_proxy; do
9     python train.py bands.combo=$COMBO data.cv_mode=holdout
10 done
11
12 # 4. Generate submission from best model
13 python generate_submission.py \
14     --results_dir results/split_window_unet \
15     --bands split_window --model unet \
16     --output submission.zip
17
18 # 5. Try a timm encoder (e.g. ResNet18)
19 python train.py model.name=timm_unet model.encoder=resnet18 bands.combo=split_window
20

```

The code has no custom CUDA kernels and no framework-specific tricks. It runs on a single GPU. It also runs on a 4 GB VRAM GPU).

## 8. Key Takeaways

**Band selection matters more than model size.** The difference between visible bands (1.286) and split-window bands (0.757) is significant.

There are no comments available.



お問い合わせ  
せ

会社  
報

© 2024 Solafune, Inc. All rights reserved

[プライバシーポリシー](#) [利用規約](#)









